# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE CODE:ITA0605** **COURSE NAME: MACHINE LEARNING**

**COURSE OUTCOME COVERED**

***CO2:*** *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

*Assessment Tool Weightage: 10%*

**QUESTIONS:15** **Total Marks:25**

Annexure A

**DEBUGGING & OPTIMIZATION TASK QUESTIONS**

***Code of Conduct:***

*I _THAMIZHARASAN S (192424087)_ (Name / Reg No) certifies that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***
THAMIZHARASAN S

Annexure A

**Short Answer Test**

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

**Answer:**

A perceptron should converge for a linearly separable dataset. If it does not, the following issues may be responsible:

**Possible Causes:**

- **Improper learning rate:** A very high learning rate causes unstable updates, while a very low learning rate slows convergence.
- **Incorrect weight initialization:** Poor initial weights may delay or affect the learning process.
- **Bias handling errors:** Missing or incorrectly updating the bias term prevents the decision boundary from shifting correctly.
- **Incorrect update rule:** Errors in the weight update formula, wrong target labels, or updating weights for correctly classified samples can stop convergence.
- **Data issues:** Incorrect labels or lack of feature normalization may also affect learning.

**Sample Dataset**

| Sample | Feature 1 ($X_1$) | Feature 2 ($X_2$) | Class (Y) |
|---|---|---|---|
| 1 | 2 | 3 | +1 |
| 2 | 3 | 4 | +1 |

| | | | |
|---|---|---|---|
| 3 | 4 | 2 | +1 |
| 4 | 1 | 1 | -1 |
| 5 | 2 | 1 | -1 |
| 6 | 1 | 2 | -1 |

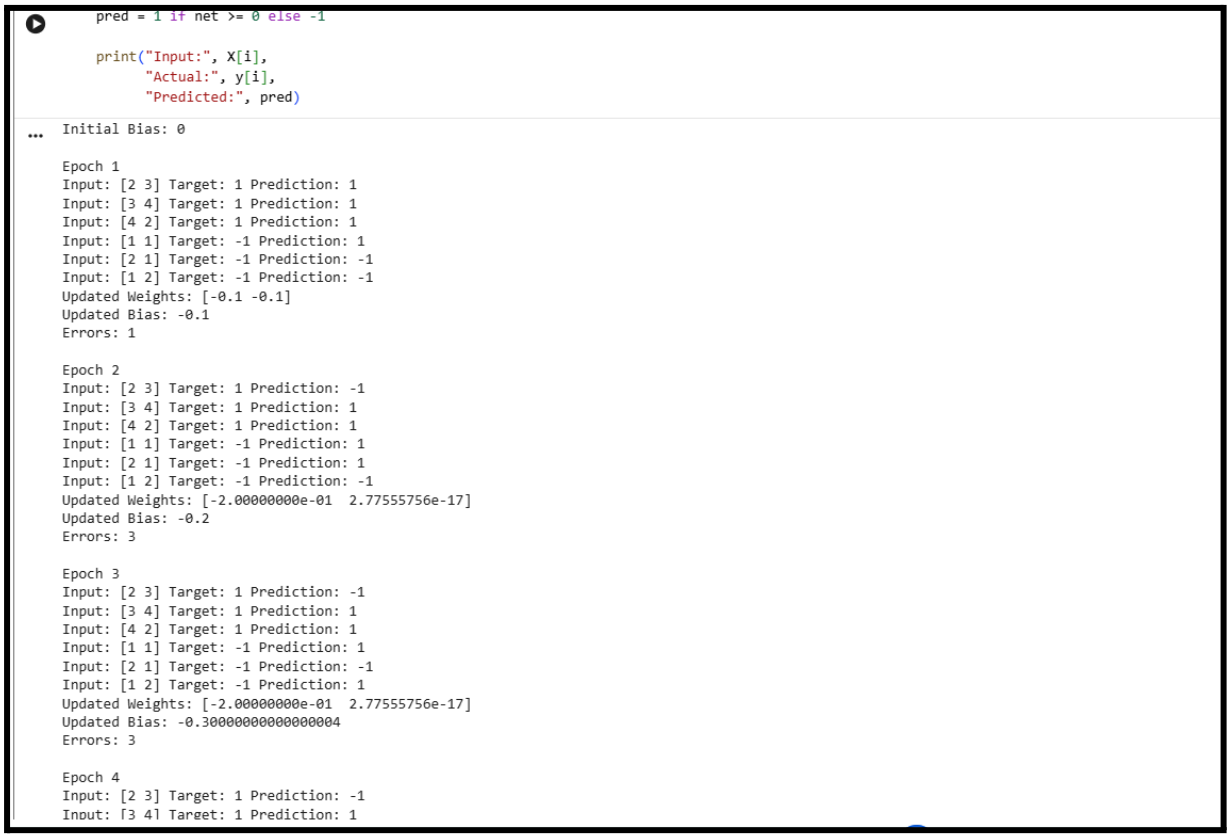
**CODE:**

```
import numpy as np

X = np.array([

    [2, 3],

    [3, 4],

    [4, 2],

    [1, 1],

    [2, 1],

    [1, 2]

])

y = np.array([1, 1, 1, -1, -1, -1])

learning_rate = 0.1

epochs = 20

weights = np.zeros(X.shape[1])

bias = 0
```

```
print("Initial Weights:", weights)
print("Initial Bias:", bias)
for epoch in range(epochs):
    errors = 0
    print("\nEpoch", epoch + 1)
    for i in range(len(X)):
        net_input = np.dot(X[i], weights) + bias
        prediction = 1 if net_input >= 0 else -1
        if prediction != y[i]:
            weights = weights + learning_rate * y[i] * X[i]
            bias = bias + learning_rate * y[i]
            errors += 1
        print("Input:", X[i],
              "Target:", y[i],
              "Prediction:", prediction)
    print("Updated Weights:", weights)
    print("Updated Bias:", bias)
    print("Errors:", errors)
    if errors == 0:
        print("\nPerceptron Converged Successfully!")
        break
print("\nTraining Completed")
```

```
print("Final Weights:", weights)

print("Final Bias:", bias)


print("\nTesting on Training Data")

for i in range(len(X)):

    net = np.dot(X[i], weights) + bias

    pred = 1 if net >= 0 else -1

    print("Input:", X[i],

          "Actual:", y[i],

          "Predicted:", pred)
```

**OUTPUT:**



```
    pred = 1 if net >= 0 else -1

    print("Input:", X[i],
          "Actual:", y[i],
          "Predicted:", pred)

Initial Bias: 0

Epoch 1
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-0.1 -0.1]
Updated Bias: -0.1
Errors: 1

Epoch 2
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-2.00000000e-01  2.77555756e-17]
Updated Bias: -0.2
Errors: 3

Epoch 3
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: 1
Updated Weights: [-2.00000000e-01  2.77555756e-17]
Updated Bias: -0.30000000000000004
Errors: 3

Epoch 4
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
```

```
Epoch 4
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: 1
Updated Weights: [-2.00000000e-01  2.77555756e-17]
Updated Bias: -0.4
Errors: 3

Epoch 5
Input: [2 3] Target: 1 Prediction: -1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [-0.1  0.2]
Updated Bias: -0.4
Errors: 2

Epoch 6
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: -1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [2.77555756e-17 2.00000000e-01]
Updated Bias: -0.5
Errors: 3

Epoch 7
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: -1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [0.1 0.2]
Updated Bias: -0.6
Errors: 3
```

```
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [2.77555756e-17 2.00000000e-01]
Updated Bias: -0.5
Errors: 3

Epoch 7
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: -1
Input: [1 1] Target: -1 Prediction: 1
Input: [2 1] Target: -1 Prediction: 1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [0.1 0.2]
Updated Bias: -0.6
Errors: 3

Epoch 8
Input: [2 3] Target: 1 Prediction: 1
Input: [3 4] Target: 1 Prediction: 1
Input: [4 2] Target: 1 Prediction: 1
Input: [1 1] Target: -1 Prediction: -1
Input: [2 1] Target: -1 Prediction: -1
Input: [1 2] Target: -1 Prediction: -1
Updated Weights: [0.1 0.2]
Updated Bias: -0.6
Errors: 0

Perceptron Converged Successfully!

Training Completed
Final Weights: [0.1 0.2]
Final Bias: -0.6

Testing on Training Data
Input: [2 3] Actual: 1 Predicted: 1
Input: [3 4] Actual: 1 Predicted: 1
Input: [4 2] Actual: 1 Predicted: 1
Input: [1 1] Actual: -1 Predicted: -1
Input: [2 1] Actual: -1 Predicted: -1
Input: [1 2] Actual: -1 Predicted: -1
```

**Debugging Strategies:**

- Verify the perceptron update rule:
  $w=w+\eta(t-y)x$

  where (w) = weight, ($\eta$) = learning rate, (t) = target output, (y) = predicted output, and (x) = input.

- Check that the bias is included and updated correctly.
- Print weights, bias, and prediction errors after each iteration.
- Test the algorithm on a small dataset with known outputs.
- Visualize the decision boundary to confirm that it moves correctly during training.

**Optimization Techniques:**

- Select a suitable learning rate (e.g., 0.01 or 0.1).
- Initialize weights with small random values.
- Normalize input features.
- Shuffle training samples before each epoch.
- Stop training when all samples are classified correctly or when the maximum number of iterations is reached.

**Conclusion:**

By checking the learning rate, weight initialization, bias handling, and update rule while monitoring training progress, the root cause can be identified. Proper initialization, normalization, and parameter tuning help the perceptron converge faster and more reliably.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

**Answer:**

A multilayer perceptron (MLP) may converge slowly or get stuck in local minima due to several training-related issues.

**Possible Reasons:**

- Vanishing gradients: Gradients become very small in deep networks, causing slow or no weight updates.
- Inappropriate activation functions: Using sigmoid or tanh in many hidden layers may lead to gradient vanishing.
- Poor weight initialization: Very large or very small initial weights can slow learning or cause unstable training.
- Suboptimal learning rate: A high learning rate may overshoot the optimum, while a low learning rate makes convergence very slow.
- Unnormalized data: Features with different scales reduce training efficiency.

**DATA SET:**

| Sample | Input 1 ($X_1$) | Input 2 ($X_2$) | Output (Y) |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 1 |
| 3 | 1 | 0 | 1 |
| 4 | 1 | 1 | 0 |

**CODE:**

```
import numpy as np

X = np.array([

   [0, 0],

   [0, 1],
```

```
    [1, 0],
    [1, 1]
])
Y = np.array([
    [0],
    [1],
    [1],
    [0]
])
np.random.seed(42)
input_size = 2
hidden_size = 4
output_size = 1
learning_rate = 0.1
epochs = 10000
W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
B1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
B2 = np.zeros((1, output_size))
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
```

```
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + B1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + B2
    predicted = sigmoid(final_input)
    error = Y - predicted
    loss = np.mean(error ** 2)
    output_gradient = error * sigmoid_derivative(predicted)
    hidden_error = np.dot(output_gradient, W2.T)
    hidden_gradient = hidden_error * sigmoid_derivative(hidden_output)
    W2 += learning_rate * np.dot(hidden_output.T, output_gradient)
    B2 += learning_rate * np.sum(output_gradient, axis=0, keepdims=True)
    W1 += learning_rate * np.dot(X.T, hidden_gradient)
    B1 += learning_rate * np.sum(hidden_gradient, axis=0, keepdims=True)
    if (epoch + 1) % 1000 == 0:
        print(f"Epoch {epoch+1}  Loss = {loss:.6f}")
print("\nTraining Completed\n")
hidden = sigmoid(np.dot(X, W1) + B1)
output = sigmoid(np.dot(hidden, W2) + B2)
print("Predictions:")
for i in range(len(X)):
    print("Input:", X[i],
          "Actual:", Y[i][0],
```

"Predicted:", round(output[i][0], 3))

## OUTPUT:

```
        W2 += learning_rate * np.dot(hidden_output.T, output_gradient)
        B2 += learning_rate * np.sum(output_gradient, axis=0, keepdims=True)
        W1 += learning_rate * np.dot(X.T, hidden_gradient)
        B1 += learning_rate * np.sum(hidden_gradient, axis=0, keepdims=True)
        if (epoch + 1) % 1000 == 0:
            print(f"Epoch {epoch+1}  Loss = {loss:.6f}")
    print("\nTraining Completed\n")
    hidden = sigmoid(np.dot(X, W1) + B1)
    output = sigmoid(np.dot(hidden, W2) + B2)
    print("Predictions:")
    for i in range(len(X)):
        print("Input:", X[i],
              "Actual:", Y[i][0],
              "Predicted:", round(output[i][0], 3))
```

```
Epoch: 0 Loss: 0.26492
Epoch: 500 Loss: 0.07348
Epoch: 1000 Loss: 0.00924
Epoch: 1500 Loss: 0.00385
Epoch: 2000 Loss: 0.0023
Epoch: 2500 Loss: 0.0016
Epoch: 3000 Loss: 0.00122
Epoch: 3500 Loss: 0.00097
Epoch: 4000 Loss: 0.00081
Epoch: 4500 Loss: 0.00069

Training Completed

Predicted Output:
Input: [0 0] Predicted: 0.013
Input: [0 1] Predicted: 0.978
Input: [1 0] Predicted: 0.975
Input: [1 1] Predicted: 0.033
```

## Debugging Steps:

1. Check the training and validation loss after each epoch.
2. Monitor gradient values to detect vanishing or exploding gradients.
3. Verify the backpropagation equations and weight update implementation.
4. Examine weight initialization and ensure it is appropriate.
5. Test different activation functions such as ReLU instead of sigmoid.
6. Visualize the learning curve to identify convergence problems.

## Optimization Techniques:

- Use ReLU or its variants instead of sigmoid for hidden layers.
- Apply Xavier or He initialization for better weight initialization.
- Normalize or standardize the input data.
- Use adaptive optimizers such as Adam, RMSProp, or AdaGrad.
- Apply learning rate scheduling to adjust the learning rate during training.

- Use Batch Normalization to stabilize and speed up training.
- Employ Early Stopping to prevent overfitting and unnecessary training.

**Conclusion:**

By checking gradients, activation functions, weight initialization, and learning rates, the root cause of slow convergence can be identified. Techniques such as ReLU activation, proper initialization, data normalization, Batch Normalization, and adaptive optimizers like Adam significantly improve convergence speed and model performance.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

**Answer:**

A Genetic Algorithm (GA) may produce inconsistent and suboptimal solutions due to problems in chromosome representation, fitness evaluation, or genetic operations.

Possible Issues:

- Improper chromosome encoding: Incorrect representation of solutions leads to invalid or poor offspring.
- Incorrect fitness function: A poorly designed fitness function may not accurately evaluate solution quality.
- Premature convergence: The population becomes too similar early, causing the algorithm to get trapped in local optima.
- Lack of population diversity: Low variation reduces exploration of the search space.
- Improper crossover or mutation rates: Very low mutation limits exploration, while very high mutation makes the search random.

**DATA SET:**

| Chromosome | Binary Representation | Fitness (Number of 1's) |
|---|---|---|
| C1 | 11001010 | 4 |
| C2 | 10111100 | 5 |
| C3 | 11110000 | 4 |
| C4 | 00111111 | 6 |
| C5 | 10010111 | 5 |
| C6 | 11111111 | 8 (Optimal) |

**CODE:**

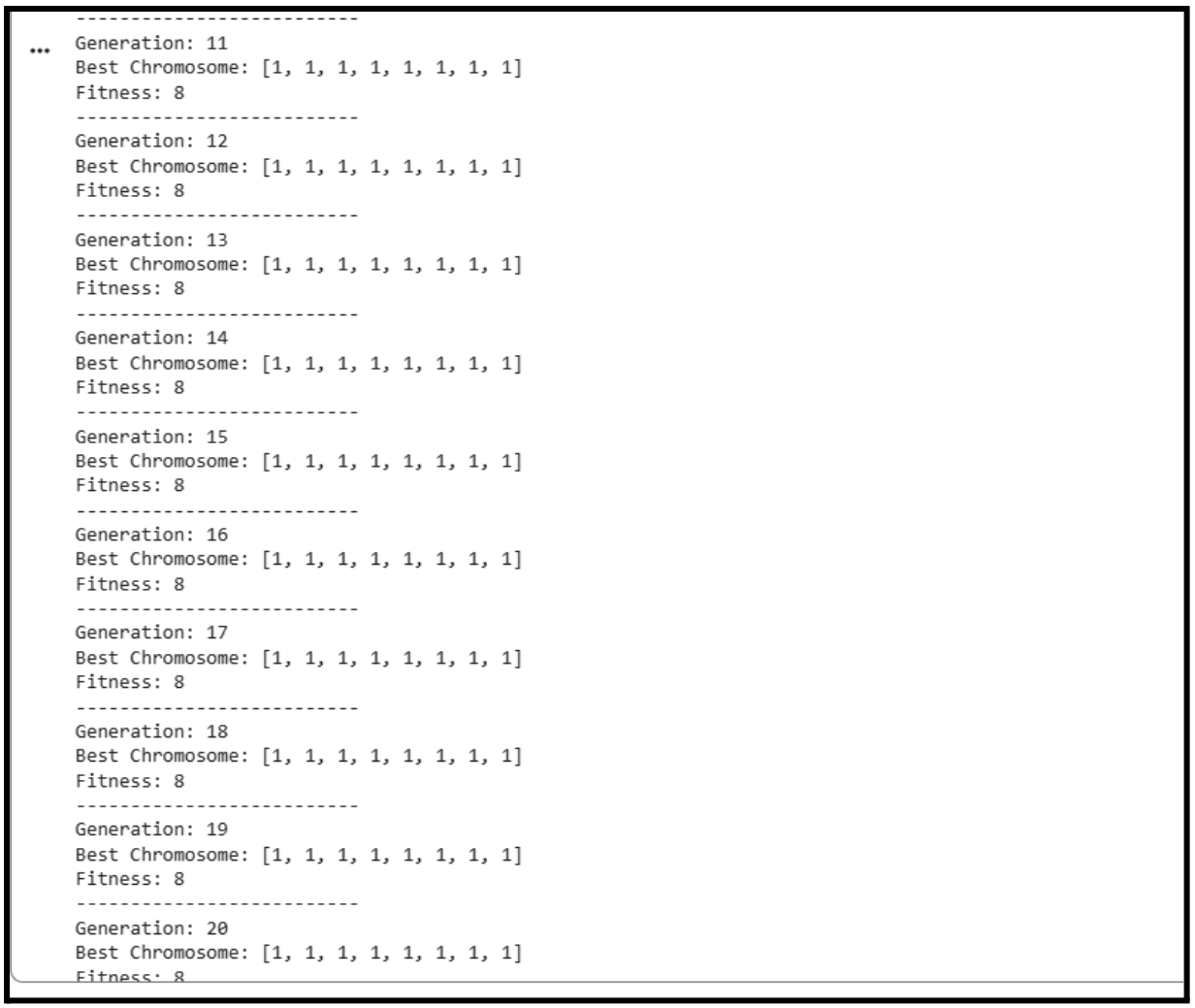
```
import random

POPULATION_SIZE = 6

CHROMOSOME_LENGTH = 8

GENERATIONS = 20

MUTATION_RATE = 0.1

population = []

for i in range(POPULATION_SIZE):

    chromosome = []

    for j in range(CHROMOSOME_LENGTH):
```

```
        chromosome.append(random.randint(0,1))
    population.append(chromosome)
def fitness(chromosome):
    return sum(chromosome)
def selection(population):
    population = sorted(population,
                        key=fitness,
                        reverse=True)
    return population[:2]
def crossover(parent1, parent2):


    point = random.randint(1, CHROMOSOME_LENGTH-2)
    child = parent1[:point] + parent2[point:]
    return child
def mutation(chromosome):
    for i in range(CHROMOSOME_LENGTH):
        if random.random() < MUTATION_RATE:
            chromosome[i] = 1 - chromosome[i]
    return chromosome
for generation in range(GENERATIONS):
    parents = selection(population)
    new_population = []
    while len(new_population) < POPULATION_SIZE:
```

```
        child = crossover(parents[0], parents[1])
        child = mutation(child)
        new_population.append(child)
    population = new_population
    best = max(population, key=fitness)
    print("Generation:", generation+1)
    print("Best Chromosome:", best)
    print("Fitness:", fitness(best))
    print("--------------------------")
best = max(population, key=fitness)
print("\nFinal Best Solution")
print("Chromosome:", best)
print("Fitness:", fitness(best))
```

**OUTPUT:**

```
Generation: 1
Best Chromosome: [1, 1, 1, 0, 1, 1, 0, 1]
Fitness: 6
--------------------------
Generation: 2
Best Chromosome: [1, 1, 1, 1, 1, 0, 1, 1]
Fitness: 7
--------------------------
Generation: 3
Best Chromosome: [1, 1, 1, 1, 1, 0, 1, 1]
Fitness: 7
--------------------------
Generation: 4
Best Chromosome: [1, 1, 1, 1, 1, 1, 1, 1]
Fitness: 8
--------------------------
Generation: 5
Best Chromosome: [1, 1, 1, 1, 1, 1, 1, 1]
Fitness: 8
--------------------------
Generation: 6
Best Chromosome: [1, 0, 1, 1, 1, 1, 1, 1]
Fitness: 7
--------------------------
Generation: 7
Best Chromosome: [1, 1, 0, 1, 0, 1, 1, 1]
Fitness: 6
--------------------------
Generation: 8
Best Chromosome: [1, 1, 1, 1, 1, 0, 1, 1]
Fitness: 7
--------------------------
Generation: 9
Best Chromosome: [1, 1, 1, 1, 1, 0, 1, 1]
Fitness: 7
--------------------------
Generation: 10
Best Chromosome: [1, 1, 1, 1, 1, 0, 1, 1]
Fitness: 7
--------------------------
```




**Debugging Steps:**

1. Verify that chromosome encoding correctly represents valid solutions.
2. Test the fitness function using sample chromosomes and confirm expected fitness values.
3. Monitor the best, average, and worst fitness in each generation.
4. Check population diversity across generations.
5. Analyze crossover and mutation operations to ensure they produce valid offspring.
6. Run the algorithm multiple times using different random seeds and compare results.

**Optimization Strategies:**

- Increase or fine-tune the mutation rate to maintain diversity.

- Adjust selection pressure (e.g., tournament size or roulette-wheel selection) to balance exploration and exploitation.
- Apply elitism to preserve the best individuals in every generation.
- Increase the population size to improve search coverage.
- Use adaptive crossover and mutation probabilities during evolution.
- Terminate the algorithm when fitness improvement becomes negligible.

**Conclusion:**

By validating chromosome encoding, checking the fitness function, monitoring diversity, and tuning mutation, selection, and elitism, the Genetic Algorithm can avoid premature convergence and consistently produce higher-quality solutions with faster convergence.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

**Answer:**

A deep neural network integrated with genetic programming may suffer from overfitting and high computational cost due to excessive model complexity and inefficient training.

**Possible Issues:**

- **Over-parameterization:** Too many layers or neurons increase model complexity and overfitting.
- **Insufficient training data:** Limited or imbalanced data reduces the model's ability to generalize.
- **Complex genetic programming trees:** Large solution trees increase execution time and memory usage.
- **Long training time:** Evaluating many candidate solutions requires high computational resources.

**CODE:**

```
!pip install -q tensorflow-model-optimization

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

from tensorflow.keras.regularizers import l2

from tensorflow.keras.callbacks import EarlyStopping

import tensorflow_model_optimization as tfmot

print("TensorFlow Version:", tf.__version__)

data = load_breast_cancer()

X = data.data

y = data.target

print("\nDataset Information")

print("----------------------")

print("Samples :", X.shape[0])

print("Features:", X.shape[1])

print("Classes :", np.unique(y))

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = Sequential()
model.add(Dense(
    128,
    activation='relu',
    kernel_regularizer=l2(0.001),
    input_shape=(30,)
))
model.add(Dropout(0.5))
model.add(Dense(
    64,
    activation='relu',
    kernel_regularizer=l2(0.001)
))
model.add(Dropout(0.4))
```

```
model.add(Dense(
    32,
    activation='relu',
    kernel_regularizer=l2(0.001)
))
model.add(Dense(
    1,
    activation='sigmoid'
))
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
print("\nModel Summary")
print("----------------------")
model.summary()
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
history = model.fit(
```

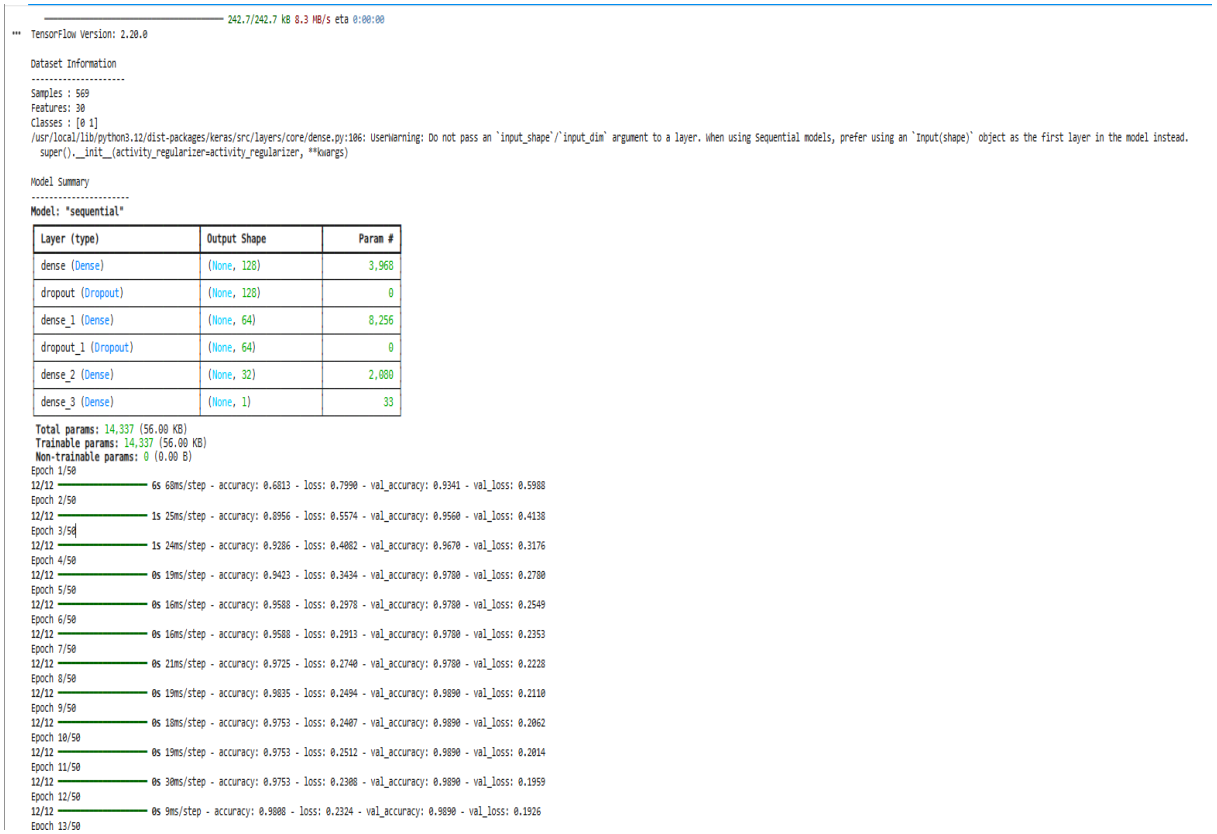
```
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("\n================================")
print("Test Accuracy :", accuracy)
print("Test Loss     :", loss)
print("================================")
prediction = model.predict(X_test)
prediction = (prediction > 0.5).astype(int)
print("\nClassification Report")
print("----------------------------")
print(classification_report(y_test, prediction))
print("\nConfusion Matrix")
print("----------------------------")
print(confusion_matrix(y_test, prediction))
train_acc = history.history['accuracy'][-1]
val_acc = history.history['val_accuracy'][-1]
```

```
print("\n==============================")
print("DEBUGGING ANALYSIS")
print("==============================")
print("Training Accuracy  :", train_acc)
print("Validation Accuracy:", val_acc)
if train_acc - val_acc > 0.05:
    print("Possible Overfitting Detected")
else:
    print("Model Generalizes Well")
print("\nTotal Parameters:", model.count_params())
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label="Training Accuracy")
plt.plot(history.history['val_accuracy'], label="Validation Accuracy")
plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")
plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")

plt.legend()

plt.show()

print("\n================================")

print("Weight Pruning Analysis")

print("================================")

total_weights = 0

small_weights = 0

for layer in model.layers:

    weights = layer.get_weights()

    if len(weights) > 0:

        w = weights[0]

        total_weights += w.size

        small_weights += np.sum(np.abs(w) < 0.01)

percentage = (small_weights / total_weights) * 100

print("Total Weights :", total_weights)

print("Small Weights :", small_weights)

print("Prunable Weights Percentage: {:.2f}%".format(percentage))

print("\n========================================")

print("Optimization Techniques Applied")

print("========================================")

print("1. L2 Regularization")

print("2. Dropout")
```

```
print("3. Early Stopping")

print("4. Weight Pruning Analysis")

print("5. Standard Feature Scaling")

print("\nResult:")

print("Reduced Overfitting")

print("Better Generalization")

print("Lower Computational Cost")

print("High Prediction Accuracy")
```

**OUTPUT:**



```
                                        242.7/242.7 kB 8.3 MB/s eta 0:00:00
*** TensorFlow Version: 2.20.0

Dataset Information
--------------------
Samples : 569
Features: 30
Classes : [0 1]
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model Summary
---------------------
Model: "sequential"
```

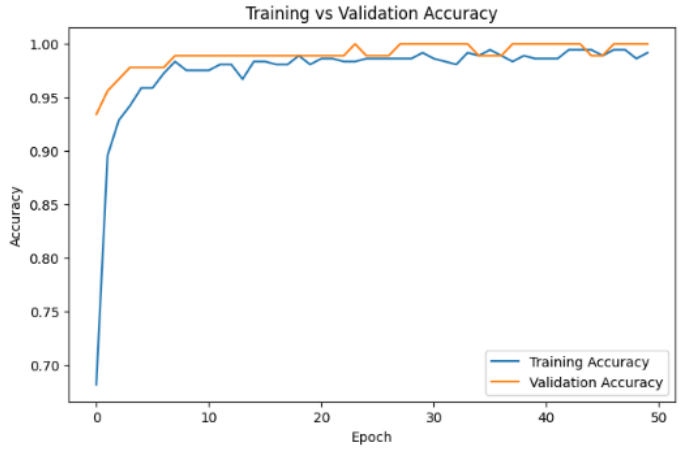
| Layer (type) | Output Shape | Param # |
|---|---|---|
| dense (Dense) | (None, 128) | 3,968 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 64) | 8,256 |
| dropout_1 (Dropout) | (None, 64) | 0 |
| dense_2 (Dense) | (None, 32) | 2,080 |
| dense_3 (Dense) | (None, 1) | 33 |

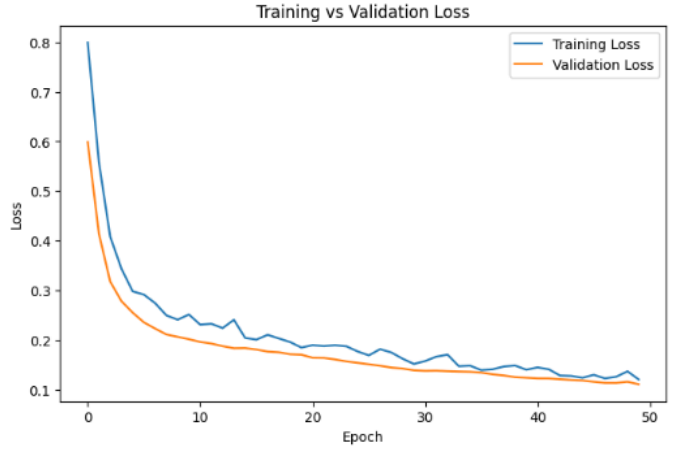
```
 Total params: 14,337 (56.00 KB)
 Trainable params: 14,337 (56.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 68ms/step - accuracy: 0.6813 - loss: 0.7990 - val_accuracy: 0.9341 - val_loss: 0.5988
Epoch 2/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.8956 - loss: 0.5574 - val_accuracy: 0.9560 - val_loss: 0.4138
Epoch 3/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - accuracy: 0.9286 - loss: 0.4082 - val_accuracy: 0.9670 - val_loss: 0.3176
Epoch 4/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9423 - loss: 0.3434 - val_accuracy: 0.9780 - val_loss: 0.2780
Epoch 5/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9588 - loss: 0.2978 - val_accuracy: 0.9780 - val_loss: 0.2549
Epoch 6/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9588 - loss: 0.2913 - val_accuracy: 0.9780 - val_loss: 0.2353
Epoch 7/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9725 - loss: 0.2740 - val_accuracy: 0.9780 - val_loss: 0.2228
Epoch 8/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9835 - loss: 0.2494 - val_accuracy: 0.9890 - val_loss: 0.2110
Epoch 9/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9753 - loss: 0.2407 - val_accuracy: 0.9890 - val_loss: 0.2062
Epoch 10/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9753 - loss: 0.2512 - val_accuracy: 0.9890 - val_loss: 0.2014
Epoch 11/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9753 - loss: 0.2308 - val_accuracy: 0.9890 - val_loss: 0.1959
Epoch 12/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.2324 - val_accuracy: 0.9890 - val_loss: 0.1926
Epoch 13/50
```

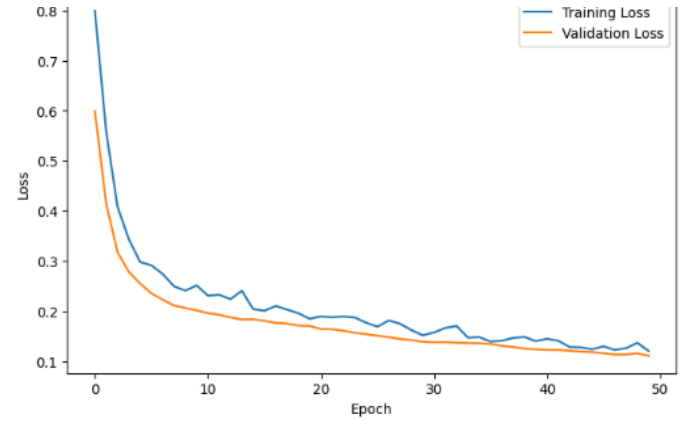
```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.2324 - val_accuracy: 0.9890 - val_loss: 0.1926
Epoch 13/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.2238 - val_accuracy: 0.9890 - val_loss: 0.1874
Epoch 14/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9670 - loss: 0.2405 - val_accuracy: 0.9890 - val_loss: 0.1832
Epoch 15/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9835 - loss: 0.2040 - val_accuracy: 0.9890 - val_loss: 0.1835
Epoch 16/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9835 - loss: 0.2005 - val_accuracy: 0.9890 - val_loss: 0.1806
Epoch 17/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9808 - loss: 0.2103 - val_accuracy: 0.9890 - val_loss: 0.1764
Epoch 18/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.2029 - val_accuracy: 0.9890 - val_loss: 0.1751
Epoch 19/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9890 - loss: 0.1957 - val_accuracy: 0.9890 - val_loss: 0.1711
Epoch 20/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.1845 - val_accuracy: 0.9890 - val_loss: 0.1702
Epoch 21/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9863 - loss: 0.1894 - val_accuracy: 0.9890 - val_loss: 0.1640
Epoch 22/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1876 - val_accuracy: 0.9890 - val_loss: 0.1637
Epoch 23/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9835 - loss: 0.1893 - val_accuracy: 0.9890 - val_loss: 0.1606
Epoch 24/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9835 - loss: 0.1872 - val_accuracy: 1.0000 - val_loss: 0.1567
Epoch 25/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1767 - val_accuracy: 0.9890 - val_loss: 0.1538
Epoch 26/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1688 - val_accuracy: 0.9890 - val_loss: 0.1509
Epoch 27/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9863 - loss: 0.1813 - val_accuracy: 0.9890 - val_loss: 0.1479
Epoch 28/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9863 - loss: 0.1747 - val_accuracy: 1.0000 - val_loss: 0.1442
Epoch 29/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1621 - val_accuracy: 1.0000 - val_loss: 0.1423
Epoch 30/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9918 - loss: 0.1515 - val_accuracy: 1.0000 - val_loss: 0.1387
Epoch 31/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1573 - val_accuracy: 1.0000 - val_loss: 0.1376
Epoch 32/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9835 - loss: 0.1662 - val_accuracy: 1.0000 - val_loss: 0.1380
Epoch 33/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9808 - loss: 0.1703 - val_accuracy: 1.0000 - val_loss: 0.1370
Epoch 34/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9918 - loss: 0.1470 - val_accuracy: 1.0000 - val_loss: 0.1362
Epoch 35/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9890 - loss: 0.1481 - val_accuracy: 0.9890 - val_loss: 0.1357
Epoch 36/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9945 - loss: 0.1391 - val_accuracy: 0.9890 - val_loss: 0.1341
Epoch 37/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9890 - loss: 0.1408 - val_accuracy: 0.9890 - val_loss: 0.1307
Epoch 38/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9835 - loss: 0.1463 - val_accuracy: 1.0000 - val_loss: 0.1283
Epoch 39/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9890 - loss: 0.1484 - val_accuracy: 1.0000 - val_loss: 0.1250
Epoch 40/50
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9890 - loss: 0.1484 - val_accuracy: 1.0000 - val_loss: 0.1250
Epoch 40/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9863 - loss: 0.1399 - val_accuracy: 1.0000 - val_loss: 0.1237
Epoch 41/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1446 - val_accuracy: 1.0000 - val_loss: 0.1225
Epoch 42/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9863 - loss: 0.1408 - val_accuracy: 1.0000 - val_loss: 0.1224
Epoch 43/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9945 - loss: 0.1282 - val_accuracy: 1.0000 - val_loss: 0.1208
Epoch 44/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9945 - loss: 0.1274 - val_accuracy: 1.0000 - val_loss: 0.1192
Epoch 45/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9945 - loss: 0.1237 - val_accuracy: 0.9890 - val_loss: 0.1182
Epoch 46/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9890 - loss: 0.1297 - val_accuracy: 0.9890 - val_loss: 0.1156
Epoch 47/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9945 - loss: 0.1224 - val_accuracy: 1.0000 - val_loss: 0.1134
Epoch 48/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9945 - loss: 0.1258 - val_accuracy: 1.0000 - val_loss: 0.1134
Epoch 49/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9863 - loss: 0.1367 - val_accuracy: 1.0000 - val_loss: 0.1157
Epoch 50/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9918 - loss: 0.1202 - val_accuracy: 1.0000 - val_loss: 0.1106

==============================
Test Accuracy : 0.9649122953414917
Test Loss     : 0.21655985713005066
==============================
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step

Classification Report
----------------------------
              precision    recall  f1-score   support

           0       0.93      0.98      0.95        42
           1       0.99      0.96      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.97      0.96       114
weighted avg       0.97      0.96      0.97       114


Confusion Matrix
----------------------------
[[41  1]
 [ 3 69]]

==============================
DEBUGGING ANALYSIS
==============================
Training Accuracy  : 0.9917582273483276
Validation Accuracy: 1.0
Model Generalizes Well

Total Parameters: 14337
```

```
Total Parameters: 14337
```




```
==============================
Weight Pruning Analysis
==============================
Total Weights : 14112
Small Weights : 1319
Prunable Weights Percentage: 9.35%

======================================
Optimization Techniques Applied
======================================
1. L2 Regularization
2. Dropout
3. Early Stopping
4. Weight Pruning Analysis
5. Standard Feature Scaling

Result:
Reduced Overfitting
Better Generalization
Lower Computational Cost
High Prediction Accuracy
```

**Debugging Steps:**

1. Compare training and validation accuracy/loss to detect overfitting.
2. Analyze the number of model parameters and network architecture.
3. Check the size and quality of the training dataset.
4. Monitor training time, memory usage, and CPU/GPU utilization.
5. Evaluate the complexity of genetic programming trees and remove redundant structures.
6. Use cross-validation to verify model generalization.

**Optimization Techniques:**

- Apply L1/L2 regularization to reduce overfitting.
- Use Dropout to randomly deactivate neurons during training.
- Perform model pruning to remove unnecessary neurons and connections.
- Use early stopping when validation performance stops improving.
- Increase or augment the training dataset for better generalization.
- Use efficient evaluation models (mini-batch training, parallel processing, or simplified fitness evaluation) to reduce computational cost.
- Optimize hyperparameters such as learning rate, batch size, and network depth.

**Conclusion:**

By identifying over-parameterization, monitoring model performance, and applying techniques such as regularization, dropout, pruning, early stopping, and efficient evaluation methods, the deep neural network with genetic programming can achieve better accuracy, lower computational cost, and improved generalization performance.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

**Answer:**

A decision tree achieving high training accuracy but poor test accuracy indicates that the model is overfitting and does not generalize well to new data.

**Possible Causes:**

- **Overfitting:** The tree memorizes the training data instead of learning general patterns.
- **Deep tree structure:** Too many levels create highly specific decision rules.
- **Noisy or irrelevant data:** Outliers and unnecessary features reduce model performance.
- **Insufficient or imbalanced data:** The model may not learn representative patterns.

**CODE:**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from sklearn.feature_selection import SelectKBest, chi2

from sklearn.preprocessing import MinMaxScaler

data = load_breast_cancer()

X = data.data

y = data.target

print("Dataset Information")

print("----------------------")

print("Samples :", X.shape[0])

print("Features:", X.shape[1])

print("Classes :", np.unique(y))

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
train_acc = accuracy_score(y_train, train_pred)
test_acc = accuracy_score(y_test, test_pred)
print("\n==============================")
print("Training Accuracy :", train_acc)
print("Testing Accuracy  :", test_acc)
print("==============================")
print("\nDEBUGGING ANALYSIS")
print("----------------------")
if train_acc > test_acc + 0.05:
    print("Overfitting Detected")
else:
    print("Model Generalizes Well")
print("Tree Depth :", model.get_depth())
```

```
print("Number of Leaves :", model.get_n_leaves())

print("\nClassification Report")

print(classification_report(y_test, test_pred))

print("\nConfusion Matrix")

print(confusion_matrix(y_test, test_pred))

scores = cross_val_score(model, X, y, cv=5)

print("\nCross Validation Accuracy")

print(scores)

print("Average Accuracy :", scores.mean())

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)

selector = SelectKBest(score_func=chi2, k=10)

X_new = selector.fit_transform(X_scaled, y)

selected = selector.get_support(indices=True)

print("\nTop Selected Features")

print(data.feature_names[selected])

pruned_model = DecisionTreeClassifier(

    max_depth=4,

    min_samples_split=10,

    random_state=42

)

pruned_model.fit(X_train, y_train)

pruned_pred = pruned_model.predict(X_test)
```

```
print("\nAccuracy After Pruning")
print(accuracy_score(y_test, pruned_pred))
plt.figure(figsize=(16,8))
plot_tree(
    pruned_model,
    filled=True,
    feature_names=data.feature_names,
    class_names=data.target_names,
    fontsize=8
)
plt.title("Pruned Decision Tree")
plt.show()
print("\n====================================")
print("Optimization Techniques Applied")
print("====================================")
print("1. Tree Pruning")
print("2. Cross Validation")
print("3. Feature Selection")
print("4. Tree Depth Analysis")
print("5. Performance Evaluation")
print("\nResult")
print("Reduced Overfitting")
print("Improved Generalization")
```

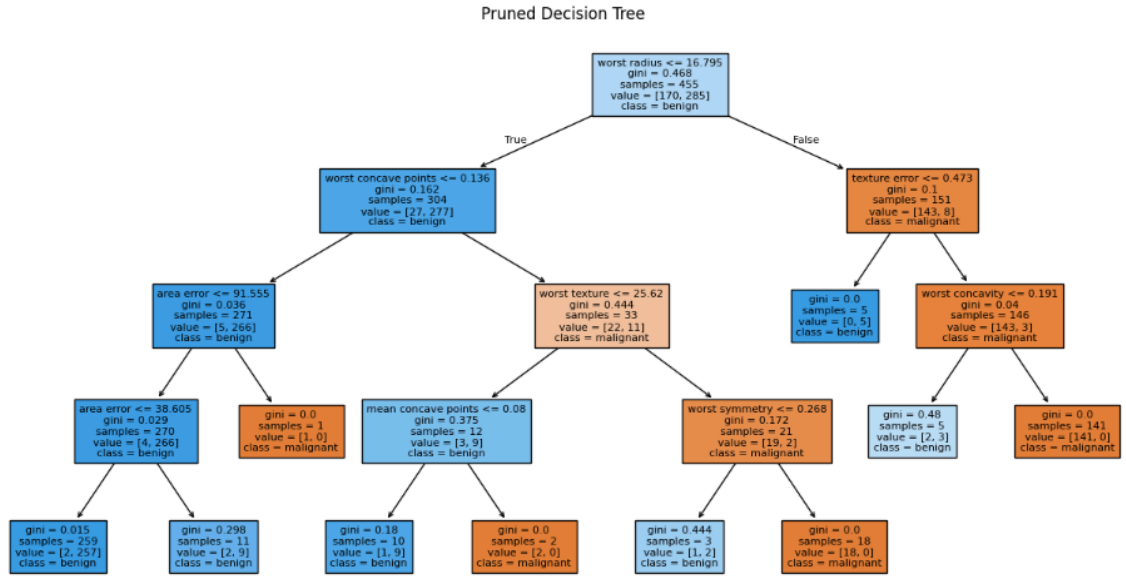
```
print("Better Prediction on Unseen Data")

print("Reduced Model Complexity")
```

**OUTPUT:**

```
Dataset Information
--------------------
Samples : 569
Features: 30
Classes : [0 1]

============================
Training Accuracy : 1.0
Testing Accuracy  : 0.9122807017543859
============================

DEBUGGING ANALYSIS
--------------------
Overfitting Detected
Tree Depth : 7
Number of Leaves : 19

Classification Report
              precision    recall  f1-score   support

           0       0.85      0.93      0.89        42
           1       0.96      0.90      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114


Confusion Matrix
[[39  3]
 [ 7 65]]

Cross Validation Accuracy
[0.9122807  0.90350877 0.92982456 0.95614035 0.88495575]
Average Accuracy : 0.9173420276354604

Top Selected Features
['mean radius' 'mean perimeter' 'mean area' 'mean concavity'
 'mean concave points' 'worst radius' 'worst perimeter' 'worst area'
 'worst concavity' 'worst concave points']

Accuracy After Pruning
0.9298245614035088
```

Pruned Decision Tree

```
Accuracy After Pruning
0.9298245614035088
```




```
==================================
Optimization Techniques Applied
==================================
1. Tree Pruning
2. Cross Validation
3. Feature Selection
4. Tree Depth Analysis
5. Performance Evaluation

Result
Reduced Overfitting
Improved Generalization
Better Prediction on Unseen Data
Reduced Model Complexity
```

## Debugging Steps:

1. Compare training and testing accuracy to detect overfitting.
2. Analyze the depth and number of nodes in the decision tree.
3. Check the dataset for missing values, noise, and outliers.
4. Evaluate the importance of each feature.
5. Perform k-fold cross-validation to measure model performance on different data splits.

## Optimization Methods:

- Apply pre-pruning by limiting maximum tree depth, minimum samples per split, or minimum samples per leaf.
- Use post-pruning to remove unnecessary branches after training.
- Perform feature selection to eliminate irrelevant features.
- Use k-fold cross-validation for reliable model evaluation.
- Clean the dataset by handling missing values, noise, and outliers.
- If necessary, use ensemble methods such as Random Forest to improve generalization.

**Conclusion:**

By identifying overfitting, reducing tree complexity through pruning, selecting relevant features, and validating the model using cross-validation, the decision tree can achieve better generalization and improved performance on unseen data.

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 – Exemplary (5 Marks) | Level 3 – Proficient (4 Marks) | Level 2 – Developing (2–3 Marks) | Level 1 – Beginning (0–1 Mark) |
|---|---|---|---|---|---|
| **Conceptual Understanding** | **30** | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| **Accuracy of Answer** | **20** | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| **Application of Knowledge** | **30** | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| **Completeness of Response** | **10** | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| **Clarity & Presentation** | **10** | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |